

**САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА
ВЕЛИКОГО**

**Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии**

УТВЕРЖДАЮ
И. О. директора ВШ ПИ
П.Д. Дробинцев
«_____» _____ г.

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

Алгоритмизация и программирование

Тема: Поиск автоморфных чисел на отрезке $[a, b]$

Выполнил студент гр. 5130904/50001 _____

Бахуров А.С.

Старший преподаватель _____

Самочадина Т.Н.

Санкт-Петербург
2025 г.

Оглавление

Введение	3
1. Основная часть	4
1.1. Особенности реализации	4
1.2. Описание алгоритма поиска автоморфных чисел	4
1.3. Блок-схема функции isAutomorphic	4
1.4. Блок-схема алгоритма	5
Заключение	7
Список литературы	8
Приложение А	9

Введение

Аutomорфные числа представляют собой интересный класс чисел в теории чисел, обладающих следующим свойством: квадрат числа оканчивается на само это число. Классическими примерами автоморфных чисел являются 5, 6, 25, 76 и другие. Например, $5^2 = 25$ (оканчивается на 5), $25^2 = 625$ (оканчивается на 25).

Целью данной лабораторной работы является разработка программы на языке C++ для поиска всех автоморфных чисел на заданном отрезке $[a, b]$. Программа должна корректно обрабатывать входные данные, выполнять их валидацию в соответствии с требованиями задания и выводить результаты поиска в удобном для пользователя формате.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Реализовать алгоритм проверки числа на свойство автоморфности.
2. Обеспечить корректный ввод границ отрезка с проверкой на числовой тип, целочисленность, положительность и корректность соотношения границ.
3. Реализовать перебор всех чисел на заданном отрезке с выводом найденных автоморфных чисел.
4. Разработать блок-схемы, иллюстрирующие работу основного алгоритма и функции проверки.

1. Основная часть

1.1. Особенности реализации

Программа реализована на языке C++. Входные данные (начало отрезка a и конец отрезка b) считываются из стандартного потока ввода как числа с плавающей точкой (тип `double`) для последующей проверки на корректность согласно заданию.

Алгоритм работы программы можно разделить на два основных этапа:

1. **Валидация входных данных.** Последовательно проверяются четыре условия. При невыполнении любого из них программа выводит соответствующее сообщение об ошибке и завершает работу. Для проверки на целочисленность значение переменной сравнивается с его явным приведением к целому типу `int`.
2. **Поиск автоморфных чисел.** Если все проверки пройдены успешно, переменная a (начало отрезка) приводится к целому типу `int`. Далее в цикле `while` перебираются все целые числа от a до b включительно. Для каждого числа i вызывается функция `isAutomorphic(i)`. Если функция возвращает `true`, число выводится на экран, а флаг `found` устанавливается в `true`. После завершения цикла, если флаг `found` остался равным `false`, выводится сообщение об отсутствии автоморфных чисел.

1.2. Описание алгоритма поиска автоморфных чисел

Аutomorphic называется число, десятичная запись квадрата которого оканчивается десятичной записью самого числа¹. Например, $25^2 = 625$, и 25 является окончанием числа 625.

В данной работе используется классический алгоритм проверки числа n на свойство автоморфности:

1. Вычисляется квадрат числа: $sq = n * n$.
2. В цикле, пока число n больше нуля, поразрядно сравниваются последние цифры числа n и его квадрата sq .
3. Если на каком-либо шаге цифры не совпадают, число не является автоморфным.
4. Если все цифры числа n совпали с соответствующими младшими цифрами квадрата, число считается автоморфным.

1.3. Блок-схема функции `isAutomorphic`

На рисунке 1 представлена блок-схема функции `isAutomorphic`.

¹Определение автоморфного числа.

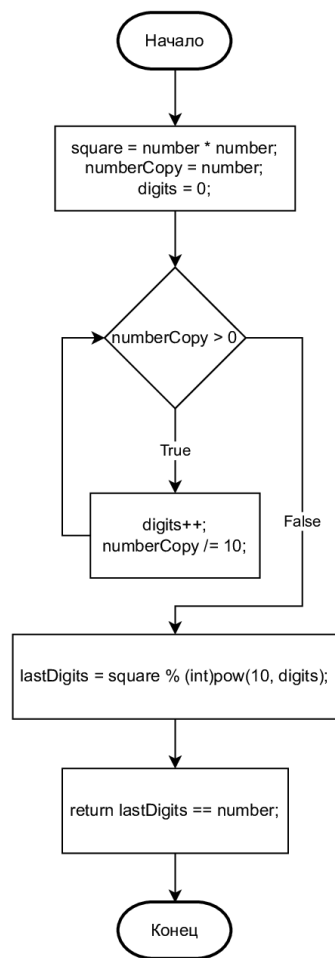


Рисунок 1 - Блок-схема функции isAutomorphic

1.4. Блок-схема алгоритма

На рисунке 2 представлена блок-схема основного алгоритма программы, включающая этапы валидации данных и поиска автоморфных чисел.

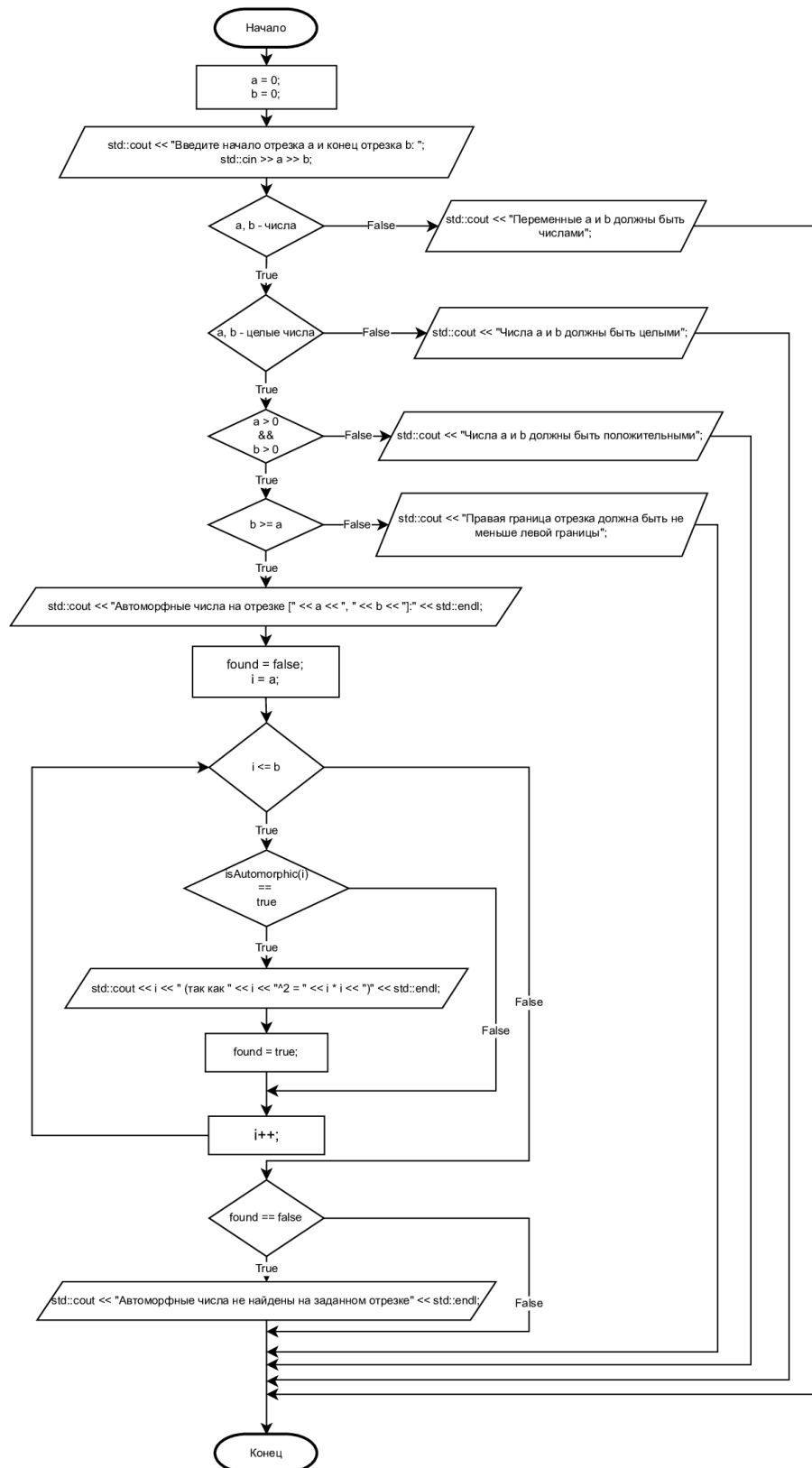


Рисунок 2 - Блок-схема алгоритма поиска автоморфных чисел

Заключение

В ходе выполнения лабораторной работы была разработана программа на языке C++ для поиска автоморфных чисел на заданном отрезке. Программа успешно проходит все тесты, предусмотренные в задании. Реализована полная валидация входных данных, что исключает возможность некорректной работы. Алгоритм поиска автоморфных чисел является эффективным и простым для понимания. Все требования, изложенные в задании, выполнены в полном объеме.

Список литературы

1. Страуструп Б. Язык программирования C++. Специальное издание. / Б. Страуструп. — М.: Бином, 2011. — 1136 с.
2. Автоморфное число // Википедия. Свободная энциклопедия. — URL: https://en.wikipedia.org/wiki/Automorphic_number
3. Шаляпин В.В. Правила оформления студенческих выпускных работ и отчетов: метод. указания / сост.: В.В. Шаляпин. - СПб.: Изд-во Политехн. ун-та, 2013. – 44с.


```

1 int main()
2 {
3     double a = 0;
4     double b = 0;
5     std::cout << "Enter the start point a and the end point b: ";
6     std::cin >> a >> b;
7     if (std::cin.fail()) {
8         std::cout << "Variables a and b must be numbers";
9     }
10    else if (a != static_cast<int>(a) || b != static_cast<int>(b)) {
11        std::cout << "The numbers a and b must be integers";
12    }
13    else if (a <= 0 || b <= 0) {
14        std::cout << "The numbers a and b must be positive";
15    }
16    else if (a > b) {
17        std::cout << "The right boundary of the segment must be at
18        least as large as the left boundary";
19    }
20    else {
21        std::cout << "Automorphic numbers on a segment [" << a << ", "
22        << b << "]:" << std::endl;
23        bool found = false;
24        int i = a;
25        while (i <= b) {
26            if (isAutomorphic(i) == true) {
27                std::cout << i << " (because " << i << "^2 = " << i * i
28                << ")"
29                << std::endl;
30                found = true;
31            }
32            i++;
33        }
34        if (found == false) {
35            std::cout << "No automorphic numbers are found in the given
36            interval" << std::endl;
37        }
38    }
39    return 0;
40 }

```